

### ■ 1.5.12 Manipulating Numerical Data

|                                      |                        |
|--------------------------------------|------------------------|
| <code>Mean[data]</code>              | mean (average value)   |
| <code>Median[data]</code>            | median (central value) |
| <code>Variance[data]</code>          | variance               |
| <code>StandardDeviation[data]</code> | standard deviation     |

Basic statistics functions from the package `Statistics.m`.

Unless it was automatically loaded when you started *Mathematica*, you first need to read in the *Mathematica* statistics package.

```
In[1]:= <<Statistics.m
```

Here is some “data”.

```
In[2]:= data = {4.3, 7.2, 8.4, 5.8, 9.2, 3.9}
Out[2]= {4.3, 7.2, 8.4, 5.8, 9.2, 3.9}
```

This gives the mean of your data.

```
In[3]:= Mean[data]
Out[3]= 6.46667
```

Here is the variance.

```
In[4]:= Variance[data]
Out[4]= 3.91222
```

When you have numerical data, it is often convenient to find a simple formula that approximates it. If you plotted the data as a curve, you could think of trying to “fit” a line or a simple curve through your data. The function `Fit` is built in to *Mathematica*; you do not need to read in any packages to get it.

|                                                                                                                                  |                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <code>Fit[{y<sub>1</sub>, y<sub>2</sub>, ...}, {f<sub>1</sub>, f<sub>2</sub>, ...}, x]</code>                                    | fit the values $y_n$ to a linear combination of functions $f_i$  |
| <code>Fit[{ {x<sub>1</sub>, y<sub>1</sub>}, {x<sub>2</sub>, y<sub>2</sub>}, ...}, {f<sub>1</sub>, f<sub>2</sub>, ...}, x]</code> | fit the points $(x_n, y_n)$ to a linear combination of the $f_i$ |

Functions for fitting curves, built in to *Mathematica*.

This generates a table of the numerical values of the exponential function. `Table` will be discussed in Section 1.7.2.

```
In[5]:= data = Table[ Exp[x/5.] , {x, 7}]
Out[5]= {1.2214, 1.49182, 1.82212, 2.22554, 2.71828,
          3.32012, 4.0552}
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: [info@wolfram.com](mailto:info@wolfram.com); <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This finds a least-squares fit to data of the form  $c_1 + c_2x + c_3x^2$ .

```
In[6]:= Fit[data, {1, x, x^2}, x]
```

```
Out[6]= 1.09428 + 0.0986337 x + 0.0459482 x^2
```

This finds a fit of the form  $c_1 + c_2x + c_3x^3 + c_4x^5$ .

```
In[7]:= Fit[data, {1, x, x^3, x^5}, x]
```

```
Out[7]= 0.96806 + 0.246829 x + 0.00428281 x^3 -
0.00000657948 x^5
```

This gives a table of  $x, y$  pairs.

```
In[8]:= data = Table[{x, Exp[Sin[x]]}, {x, 0., 1., 0.2}]
```

```
Out[8]= {{0., 1.}, {0.2, 1.21978}, {0.4, 1.47612},
{0.6, 1.75882}, {0.8, 2.04901}, {1., 2.31978}}
```

This finds a fit to the new data, of the form  $c_1 + c_2 \sin(x) + c_3 \sin(2x)$ .

```
In[9]:= Fit[%, {1, Sin[x], Sin[2x]}, x]
```

```
Out[9]= 0.989559 + 2.04199 Sin[x] - 0.418176 Sin[2 x]
```

One common way of picking out “signals” in numerical data is to find the Fourier transform or “frequency spectrum” of the data.

|                                   |                             |
|-----------------------------------|-----------------------------|
| <code>Fourier[data]</code>        | numerical Fourier transform |
| <code>InverseFourier[data]</code> | inverse Fourier transform   |

Fourier transforms.

Here is a simple square pulse.

```
In[10]:= data = {1, 1, 1, 1, -1, -1, -1, -1}
```

```
Out[10]= {1, 1, 1, 1, -1, -1, -1, -1}
```

This takes the Fourier transform of the pulse.

```
In[11]:= Fourier[data]
```

```
Out[11]= {0., 0.707107 + 1.70711 I, 0.,
0.707107 + 0.292893 I, 0., 0.707107 - 0.292893 I,
0., 0.707107 - 1.70711 I}
```